Unit III

3. Exception Handling and Collections

3.1 Exception in python
3.2 Exception roles
3.3 Exception Handling
3.4 Collections in Python – List, Tuples, Dictionaries, Sets

3.1 Exception in python:

An **exception** in Python is an error that occurs during program execution and interrupts the normal flow of instructions. Exceptions happen due to invalid operations such as dividing by zero, accessing unavailable files, incorrect data types, or invalid input. Instead of crashing the program, Python provides a mechanism to detect and handle these runtime errors gracefully.

Common Types of Exceptions:

1. **ZeroDivisionError –** dividing a number by zero
2. **ValueError –** invalid value type provided
3. **TypeError –** incompatible data types used
4. **FileNotFoundError –** file not available
5. **IndexError –** accessing out-of-range list index

Handling Exceptions:

Python uses `**try–except**` blocks to handle exceptions.

```
try:
    a = 10 / 0
except ZeroDivisionError:
    print("Cannot divide by zero")
```

Other Keywords:

`**else**` – runs if no exception occurs
`**finally**` – runs whether an exception occurs or not
`**raise**` – used to manually raise an exception

Conclusion:

Exceptions make programs more reliable by preventing unexpected crashes and allowing developers to handle errors effectively during runtime.

3.2 Exception roles:

Exceptions play an important role in Python by helping programs handle unexpected errors during execution. Instead of stopping the program abruptly, exceptions provide a structured way to detect, manage, and respond to errors. This improves the reliability and stability of programs.

Key Roles of Exceptions:

1. **Error Detection:**
   Exceptions help identify errors such as invalid input, wrong data types, division by zero, or missing files. Python automatically raises exceptions when something goes wrong.

2. **Graceful Error Handling:**
   With `try–except` blocks, programs can handle errors without crashing. This ensures smooth user experience even when unexpected situations occur.

3. **Debugging Support:**
   Exceptions provide useful error messages and tracebacks that help programmers identify the source of bugs and fix them quickly.

4. **Program Stability and Security:**
   Handling exceptions prevents program breakdowns and protects against harmful crashes, making applications more stable and secure.

5. **Controlled Flow of Execution:**
   Exceptions allow developers to define alternative paths when errors occur. Using `else`, `finally`, and `raise`, programs can manage cleanup tasks and continue running safely.

Conclusion:
Exceptions play a crucial role in building robust Python applications by detecting runtime errors, preventing crashes, and ensuring controlled, error-resistant program execution.

3.3 Exception Handling:

Exception handling in Python is a mechanism to handle **runtime errors** (exceptions) gracefully without stopping program execution. It allows programs to continue running even when unexpected errors occur, making them more reliable and user-friendly.

Key Points:

1. **Purpose:**
   Handles errors like division by zero, invalid input, or file not found, preventing program crashes.

2. **try–except Block:**
   The code that may raise an exception is placed inside a `try` block. If an exception occurs, control moves to the `except` block to handle the error.

3. **Optional else Block:**
   The `else` block runs if no exception occurs, allowing code to execute only when the `try` block is successful.

4. **finally Block:**
   The `finally` block executes regardless of whether an exception occurs or not. It is used for cleanup tasks, such as closing files.

5. **Raise Exception:**
   The `raise` keyword allows developers to manually trigger an exception when certain conditions are met.

Conclusion:
Exception handling ensures robust, error-resistant Python programs by managing runtime errors, providing meaningful feedback, and maintaining controlled program execution.

3.4 Collections in Python – List, Tuples, Dictionaries, Sets:

Python provides several built-in **collection types** to store and manage multiple data items efficiently. The main collection types are **List, Tuple, Dictionary, and Set**.

1. List:
A **list** is an ordered, mutable collection of items. It can store elements of different data types and

allows modification after creation.

Key Points:

1. Ordered collection of elements.
2. Mutable – elements can be added, removed, or changed.
3. Supports indexing, slicing, and iteration.
4. Created using square brackets: `mylist = [1, 2, 3]`.
5. Methods like `append()`, `remove()`, `sort()`, and `reverse()` are used for operations.

2. Tuple:

A **tuple** is an ordered, immutable collection of elements. Once created, its elements cannot be modified.

Key Points:

1. Ordered and immutable collection.
2. Can store mixed data types.
3. Supports indexing and slicing.
4. Created using parentheses: `mytuple = (1, 2, 3)`.
5. Useful for storing fixed data and ensures data integrity.

3. Dictionary:

A **dictionary** is an unordered collection of **key-value pairs**. Keys are unique and immutable, while values can be any data type.

Key Points:

1. Stores data as `key: value` pairs.
2. Mutable – values can be updated or deleted.
3. Created using curly braces: `mydict = {"name":"Ravi", "age":20}`.
4. Access values using keys: `mydict["name"]`.
5. Methods like `keys()`, `values()`, `items()`, and `update()` manage dictionary data.

4. Set:

A **set** is an unordered collection of **unique elements**. Sets do not allow duplicates and do not support indexing.

Key Points:

1. Unordered collection with unique elements.
2. Mutable – elements can be added or removed.
3. Created using curly braces or `set()`: `myset = {1, 2, 3}`.
4. Supports mathematical operations: union, intersection, difference.
5. Useful for removing duplicates and fast membership testing.

Conclusion:

Python collections allow efficient storage, organization, and manipulation of data. Choosing the right collection type (list, tuple, dictionary, or set) depends on whether data needs to be ordered, mutable, unique, or key-value based.